

StockDex — ALM

Tests

Plan de tests · Cas de test · Couverture

Jeremy Lardet © 2026

1. Plan de tests

Stratégie

StockDex adopte une pyramide de tests adaptée à son architecture backend Express / frontend React SPA :

- Tests unitaires (base) — moteur de fluctuation, drawCards, logique de transfer de trade
- Tests d'intégration (milieu) — routes Express avec une BDD SQLite in-memory (Testcontainers non requis)
- Tests E2E (sommet, optionnels) — scénario d'ouverture de booster via Playwright sur l'application déployée

Outils

- Jest — framework de test, runner, couverture
- Supertest — appels HTTP sur le serveur Express sans démarrer un vrai port
- SQLite in-memory (:memory:) — isolation des tests d'intégration
- Playwright — tests E2E (scope: Sprint 3, optionnel)

Couverture cible

Couverture de lignes ≥ 70 % sur packages/server/src. Le moteur de fluctuation et le service de trade visent 90 % (composants critiques).

Environnements

- CI (GitHub Actions) : Node 20, SQLite in-memory, pas de Socket.IO réel
- Local : identique, plus un serveur de développement pour les tests manuels

2. Cas de test — 3 fonctionnalités critiques

Fonctionnalités couvertes : Ouverture de booster (TC-01 à TC-03), Trade (TC-04 à TC-06), Fluctuation boursière (TC-07 à TC-09).

ID	Préconditions	Test	Étape	Résultat attendu	Obtenu
TC-01	Ouverture booster — succès	Joueur connecté avec ≥ 100 coins	POST /boosters/open avec JWT valide	Réponse 200, 5 cartes retournées, coins débités	✓ Simulé
TC-02	Ouverture booster — solde insuffisant	Joueur avec 50 coins, prix booster 100	POST /boosters/open	Réponse 402, message 'Solde insuffisant'	✓ Simulé
TC-03	Booster — tirage pondéré	BDD avec cartes à raretés connues	Appel interne drawCards(5) x 1000	Distribution statistiquement cohérente avec les poids	✓ Simulé
TC-04	Trade — proposition valide	Joueur A possède la carte offerte	POST /trades avec cartes valides	Trade créé (status: PENDING), Socket.IO notifie B	✓ Simulé

ID	Préconditions	Test	Étape	Résultat attendu	Obtenu
TC-05	Trade — carte non possédée	Joueur A ne possède pas la carte	POST /trades	Réponse 403, trade non créé	✓ Simulé
TC-06	Trade — acceptation atomique	Trade PENDING entre A et B	PATCH /trades/:id {status: accepted}	OwnedCard transférées atomiquement, trade COMPLETED	✓ Simulé
TC-07	Fluctuation — cours monte	Carte à currentPrice = 95, seuil haut = 100	Job de fluctuation déclenché	Prix $\geq$ 100 → rareté monte d'un niveau	✓ Simulé
TC-08	Fluctuation — cours descend	Carte à currentPrice = 5, seuil bas = 10	Job de fluctuation déclenché	Prix $\leq$ 10 → rareté rétrograde	✓ Simulé
TC-09	Socket.IO — broadcast	Serveur Express actif, client connecté	Job de fluctuation déclenché	Client reçoit l'événement price:update dans < 1s	✓ Simulé

3. Rapport de couverture

Estimation par module

Module	Couverture estimée	Justification
server/market/engine.js	90 %	Logique critique — tests unitaires exhaustifs
server/routes/trades.js	80 %	Flux principal + cas d'erreur
server/routes/boosters.js	75 %	Tirage pondéré + vérification solde
server/routes/auth.js	70 %	Login, register, vérif JWT
client/components/**	15 %	Composants React non couverts en unitaire

Les composants React ne sont pas couverts par Jest unitaire : l'interface est validée manuellement et par des tests E2E Playwright. La priorité est donnée aux composants serveur qui gèrent la logique métier et les données.